**UNIVERSITÉ ABDELMALEK ESSAADI**

Faculté des Sciences et Techniques – Tanger

Département d'Informatique

**MASTER SCIENCES ET TECHNIQUES**

**INTELLIGENCE ARTIFICIELLE ET SCIENCES DES DONNÉES**

**Module : Data Spaces, Data Integration & Semantic Interoperability**

---

# Rapport – Mini-projet

Mini-espace de données fédéré

Santé et qualité de l'air

---

**Encadré par :**
Pr. Baida Ouafae

**Réalisé par :**
*El Attari Taki Eddine*
*Bouazza Mohamed*
*El Kajdouhi Mohamed Ayman*

Année Universitaire 2026–2027

# Table des matières

**Introduction générale** **3**
**1 Notions de base** **3**
**2 Les données utilisées** **4**
2.1 Comparaison des sources de données . . . . . 4
2.2 Présentation du jeu de données . . . . . 4
**3 Architecture du projet** **5**
**4 TP 1 : Stockage fédéré** **6**
4.1 Réponses aux questions . . . . . 6
**5 TP 2 : Communication par API REST** **7**
5.1 Réponses aux questions . . . . . 7
**6 TP 3 : Calcul local et contrôle d'accès** **8**
6.1 Réponses aux questions . . . . . 9
**7 TP 4 : Gouvernance et confidentialité** **10**
7.1 Problème et solution . . . . . 10
7.2 Mise en œuvre . . . . . 10
7.3 Résultats et tests . . . . . 11
7.4 Réponses aux questions . . . . . 12
**8 Résultats et discussion** **14**
8.1 Limites . . . . . 14
**Conclusion générale** **15**
**A Glossaire des abréviations** **16**
**Références** **16**

# Introduction générale

Les données utiles ne se trouvent pas toujours au même endroit. Une direction de l'environnement connaît la pollution de l'air, alors qu'un hôpital connaît le nombre de malades. Pour savoir s'il y a un lien entre les deux, il faut croiser ces informations. Le problème, c'est que ces organisations ne peuvent pas envoyer leurs fichiers à n'importe qui, surtout quand il s'agit de santé.

Ce mini-projet présente un prototype construit autour d'une question simple :

> Les jours où l'air est très pollué, y a-t-il plus d'hospitalisations pour des problèmes respiratoires ?

Trois acteurs participent : une direction de l'environnement (A) qui possède les mesures de pollution, un hôpital (B) qui possède les admissions, et un laboratoire universitaire (C) qui veut analyser les résultats sans posséder ces données. Le prototype est construit en quatre TP, et chaque TP corrige un défaut du précédent.

Pour travailler dans des conditions réalistes, le prototype repose sur des données réelles à grande échelle : plus de 2,5 millions de lignes côté hôpital, pour l'État de São Paulo au Brésil, de 2015 à 2025.

# 1. Notions de base

Un **espace de données** permet à plusieurs organisations d'utiliser des données ensemble **sans perdre le contrôle** sur celles-ci. Ce n'est pas un grand serveur où tout le monde dépose ses fichiers : chaque organisation reste propriétaire de ses données et décide de ce qu'elle accepte. Trois notions reviennent dans tout le projet :

| **Terme** | **Ce que ça veut dire** | **Où** |
|---|---|---|
| Stockage fédéré | Les données restent dans des bases séparées, chez chaque organisation. | TP 1 |
| Traitement fédéré | Le calcul se fait là où se trouvent les données. | TP 3 et 4, à l'hôpital |
| Analyse fédérée | On combine des résultats autorisés, sans accéder aux bases. | TP 3 et 4, au labo |

Table 1.1 – Les trois notions de fédération

« Fédéré » ne veut pas dire qu'aucune information ne circule : des catégories de pollution et des moyennes circulent bien. La différence, c'est qu'on contrôle ce qui sort.

**Outils :** Python 3.13.5, SQLite (bases locales), FastAPI et Uvicorn (API), Pydantic (vérification des données reçues), Requests (client) et PyArrow (lecture des fichiers de données).

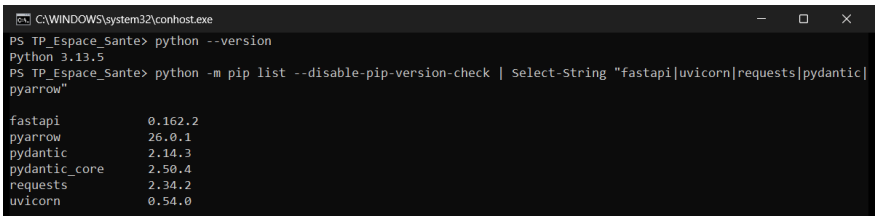



FIGURE 1.1 – Versions de Python et des bibliothèques installées

# 2. Les données utilisées

## 2.1 Comparaison des sources de données

Le projet a besoin du PM2.5 et des admissions respiratoires **par jour**, pour les mêmes villes, avec des données réelles et ouvertes. Au Maroc, quelques mesures de PM2.5 existent, mais les données de santé publiées sont des listes d'établissements et des statistiques annuelles (« Santé en chiffres ») : aucune série journalière d'admissions n'y est disponible en accès libre.

| **Source** | **Contenu** | **Adéquation au projet** |
|---|---|---|
| Maroc (data.gov.ma, ministère de la Santé) | Listes d'établissements, statistiques annuelles | Pas de série journalière d'admissions |
| Chicago NMMAPS (R) | 5 114 jours, 1987–2000 | Mesure les décès et le PM10, pas les admissions ni le PM2.5 |
| New York (NYC Open Data, EPA) | Urgences par zone postale, depuis mars 2020 | Grippe et pneumonie seulement, période marquée par la COVID-19 |
| Vitória, Brésil (Kaggle) | Environ 1 460 jours, 2015–2018 | Réel, mais peu de lignes, et seulement les enfants et les personnes âgées |
| Autres jeux Kaggle et GitHub | Jusqu'à 249 831 lignes | Admissions simulées ou origine non documentée |
| État de São Paulo (Zenodo) | 645 communes, 2015–2025, 2,6 millions de lignes | PM2.5 et admissions respiratoires réels, par jour et par commune, sous licence ouverte |

TABLE 2.1 – Comparaison des sources de données

## 2.2 Présentation du jeu de données

Le jeu de données d'Arcoverde et Maciel (2026) est publié sur Zenodo sous licence CC BY 4.0 [2]. La pollution vient de la CETESB (agence de l'environnement de São Paulo), et les hospitalisations viennent du SIH/SUS (hôpitaux publics du Brésil). Il couvre 645 communes et 4 018 jours, du 1er janvier 2015 au 31 décembre 2025. L'intégrité de l'archive est contrôlée avec son empreinte MD5.

L'archive contient aussi des fichiers avec une ligne par patient (date de naissance, sexe, code postal). Ces données personnelles ne sont pas nécessaires à l'étude et **ne sont pas utilisées**. Seules les tables agrégées, avec une ligne par commune et par jour, alimentent deux bases séparées : `MP2.5_mean` (moyenne journalière de PM2.5) et `hospitalizations_J` (hospitalisations avec un diagnostic respiratoire, codes J00 à J99).

| Base | Dossier | Lignes | Communes | Taille |
|---|---|---|---|---|
| `environnement.db` | TP 1 à 3, et TP 4 | 55 842 | 24 | 4 Mo |
| `hopital.db` | TP 1 à 3 | 2 591 610 | 645 | 173 Mo |
| `hopital.db` | TP 4 | 96 432 | 24 | 7 Mo |

TABLE 2.2 – Contenu des bases créées

**Limites.** Les deux bases viennent d'une même source publique, découpée en deux pour **simuler** deux organisations. Les données concernent seulement les hôpitaux publics. Le PM2.5 n'est pas mesuré partout : 36,6 % des admissions seulement tombent sur un jour et une commune avec une mesure. L'analyse porte sur la commune de São Paulo, car c'est la seule à disposer d'une mesure sur les 4 018 jours.

# 3. Architecture du projet

La figure 3.1 montre l'architecture finale, celle du TP 4. Chaque organisation possède sa base et son API, A et B ne se parlent pas directement, et le laboratoire ne touche jamais aux bases.

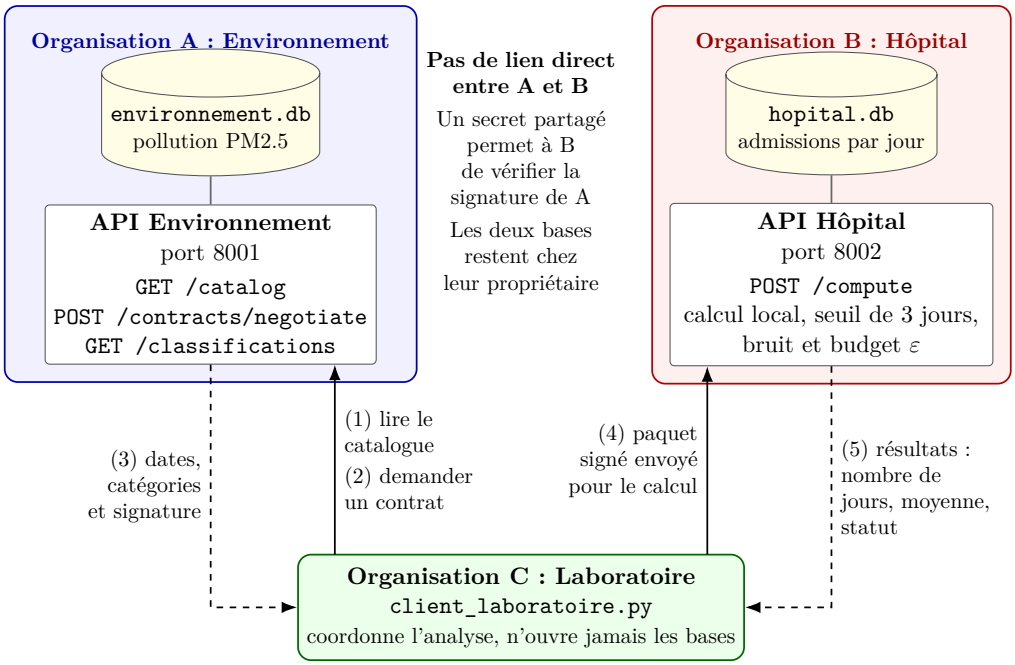



FIGURE 3.1 – Schéma d'architecture : organisations, bases, API et laboratoire

| TP | Question traitée | Le laboratoire reçoit | Protection ajoutée |
|---|---|---|---|
| 1 | Où sont les données ? | Rien | Deux bases séparées |
| 2 | Comment communiquer ? | Toutes les données brutes | Aucune |
| 3 | Comment éviter les données brutes ? | Catégories, moyennes | Clé API, calcul local |
| 4 | Comment contrôler l'usage ? | Catégories signées, moyennes | Catalogue, contrat, signature, seuil, bruit |

TABLE 3.1 – Évolution du prototype

# 4. TP 1 : Stockage fédéré

**Objectif.** Créer deux bases SQLite séparées, une par organisation.

**Mise en œuvre.** Le script `tp_federation.py` lit les fichiers de données, remplit les deux bases, puis vérifie que le nombre de lignes et le total des admissions correspondent aux fichiers d'origine. Comme une même date concerne plusieurs communes, la clé primaire est le couple `(ville, date)`. Le script `consulter_bases.py` affiche le total et les 5 premières lignes de chaque table, car la base de l'hôpital contient 2,6 millions de lignes.

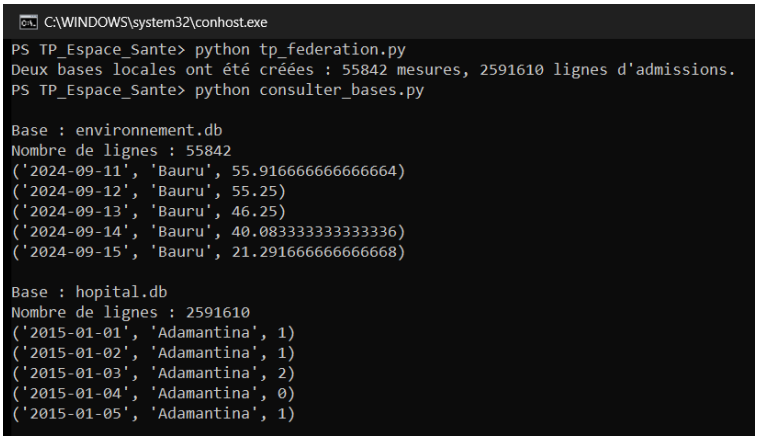



FIGURE 4.1 – Création et consultation des deux bases locales

## 4.1 Réponses aux questions

**Q1. Où sont les mesures de qualité de l'air ?** Dans `environnement.db`, table `qualite_air`, la base locale de A.

**Q2. Où sont les admissions hospitalières ?** Dans `hopital.db`, table `admissions`, la base locale de B.

**Q3. Sont-elles dans une même base ?** Non, ce sont deux fichiers SQLite indépendants.

**Q4. Comment découvrir les données sans centraliser les fichiers ?** Avec un catalogue, qui publie seulement la description des données : titre, propriétaire, période, unité, adresse d'accès et règles d'usage. C'est ce que fait `GET /catalog` au TP 4.

# 5. TP 2 : Communication par API REST

**Objectif.** Permettre aux organisations de communiquer : chaque base a sa propre API, et un client Python joue le rôle du laboratoire.

**Mise en œuvre.** L'API Environnement (port 8001) propose `GET /mesures`, et l'API Hôpital (port 8002) propose `GET /admissions`. Les deux ont aussi `GET /statistiques`. Comme la base est grande, les réponses sont paginées avec `limit` et `offset`, et le client récupère les données par paquets de 1 000 lignes. Le paramètre `ville` permet de choisir une commune.

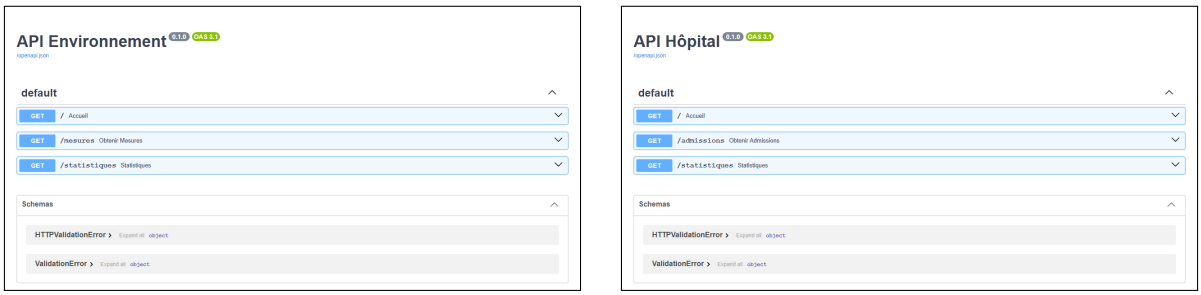



FIGURE 5.1 – Pages `/docs` des API Environnement et Hôpital (TP 2)

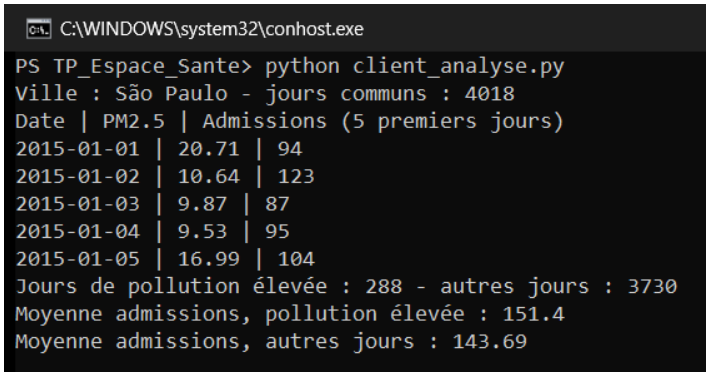



FIGURE 5.2 – Résultat du client du laboratoire (TP 2)

Sur 4 018 jours, 288 ont un PM2.5 supérieur à 30 $\mu g/m^3$. La moyenne est de 151,4 admissions ces jours-là, contre 143,69 les autres jours.

## 5.1 Réponses aux questions

**Q1. Rôle de FastAPI ?** Créer l'API : définir les adresses, vérifier les paramètres, renvoyer du JSON et générer la page de test `/docs`.

**Q2. Rôle de Uvicorn ?** C'est le serveur qui fait tourner l'application : il écoute sur un port et transmet les demandes à FastAPI.

**Q3. Pourquoi deux ports ?** Deux serveurs ne peuvent pas écouter sur le même port d'une même machine. Cela simule deux services de deux organisations.

**Q4. Le laboratoire reçoit-il les données brutes ?** Oui : les 4 018 valeurs exactes de PM2.5 et d'admissions de São Paulo.

**Q5. Une API REST garantit-elle la confidentialité ?** Non, c'est seulement un moyen de communiquer. Ici, il n'y a ni mot de passe, ni chiffrement, ni filtrage.

# 6. TP 3 : Calcul local et contrôle d'accès

**Objectif.** Éviter d'envoyer les données brutes : l'hôpital calcule lui-même, et une clé API protège l'accès.

**Mise en œuvre.** L'API Environnement ne donne plus la valeur de pollution, mais seulement la catégorie du jour (`elevee` si le PM2.5 dépasse 30 $\mu g/m^3$, sinon `non_elevee`). L'API Hôpital reçoit ces catégories, regarde dans sa base et renvoie seulement le nombre de jours et la moyenne de chaque groupe. Chaque demande doit contenir une clé dans l'en-tête `X-API-Key`.

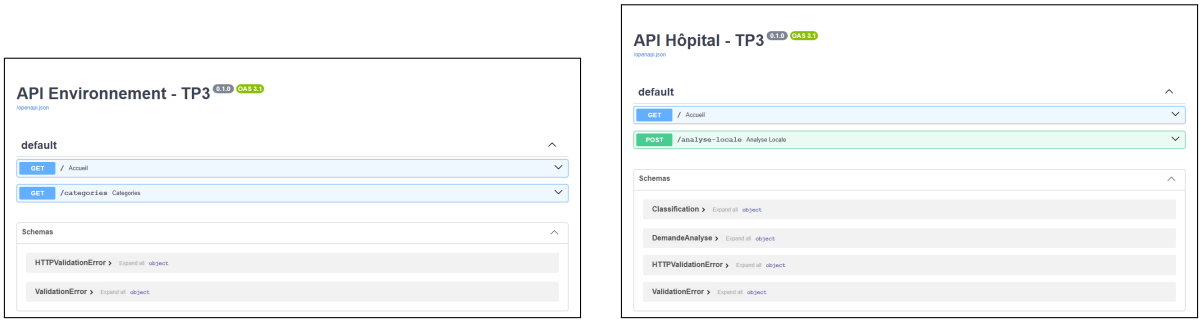



FIGURE 6.1 – Pages `/docs` des API Environnement et Hôpital (TP 3)

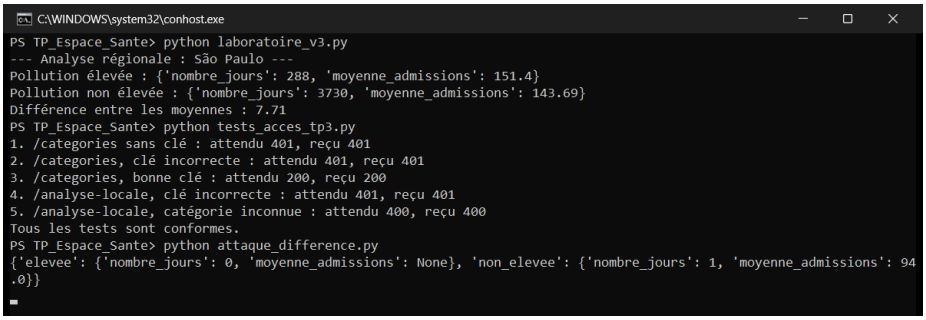



FIGURE 6.2 – Résultat du laboratoire, tests d'accès et attaque par différence (TP 3)

Les moyennes sont les mêmes qu'au TP 2 (151,4 et 143,69, soit 7,71 d'écart), mais le laboratoire ne voit plus les données de l'hôpital.

| **Test d'accès** | **Attendu** | **Obtenu** |
|---|---|---|
| `/categories` sans clé | 401 | 401 |
| `/categories` avec une clé incorrecte | 401 | 401 |
| `/categories` avec la bonne clé | 200 | 200 |
| `/analyse-locale` avec une clé incorrecte | 401 | 401 |
| `/analyse-locale` avec une catégorie inconnue | 400 | 400 |

TABLE 6.1 – Tests du contrôle d'accès (TP 3)

**Attaque par différence.** La clé contrôle **qui** appelle, mais pas **ce qui** est demandé. Le laboratoire envoie donc un groupe d'un seul jour, le 1er janvier 2015. La moyenne d'un seul jour est la valeur exacte de ce jour : l'API répond 94, et la base contient bien 94. En répétant cela, on pourrait reconstruire toute la table.

## 6.1 Réponses aux questions

**Q1. Données reçues par le laboratoire ?** TP 2 : les valeurs exactes de chaque jour. TP 3 : la catégorie de chaque jour (A), et le nombre de jours et la moyenne de chaque groupe (B).

**Q2. Pourquoi calculer à l'hôpital ?** Pour garder le contrôle : les lignes restent chez lui, seul le résultat sort.

**Q3. Informations encore transmises à l'hôpital ?** La commune, les dates et la catégorie de chaque date. Il apprend donc quels jours ont été très pollués.

**Q4. Une statistique agrégée garantit-elle la confidentialité ?** Non : avec un groupe d'un seul jour, la moyenne donne la valeur exacte (94).

**Q5. Une clé API suffit-elle pour la souveraineté ?** Non : elle dit qui peut appeler, mais pas pourquoi, et elle ne limite ni les demandes ni l'usage après la réponse.

**Q6. Comment limiter les demandes aux opérations autorisées ?** En acceptant seulement des demandes prévues et vérifiées : liste signée par A, contrat lié à une finalité, taille minimale de groupe, nombre limité de demandes.

**Q7. Composants manquants pour un véritable espace de données ?** Catalogue standard (DCAT), politiques d'usage (ODRL), contrats, identités fiables, connecteurs, vocabulaire commun, journaux d'accès, HTTPS, gestion des secrets.

# 7. TP 4 : Gouvernance et confidentialité

## 7.1 Problème et solution

Le TP 3 laisse trois failles : le laboratoire peut choisir ses groupes de dates, l'hôpital ne sait pas si la demande est légitime, et rien ne permet de découvrir les données. La solution : A publie un catalogue, vérifie la finalité, crée un contrat et signe sa liste. B refuse toute liste mal signée, ne publie pas les petits groupes et peut ajouter du bruit.

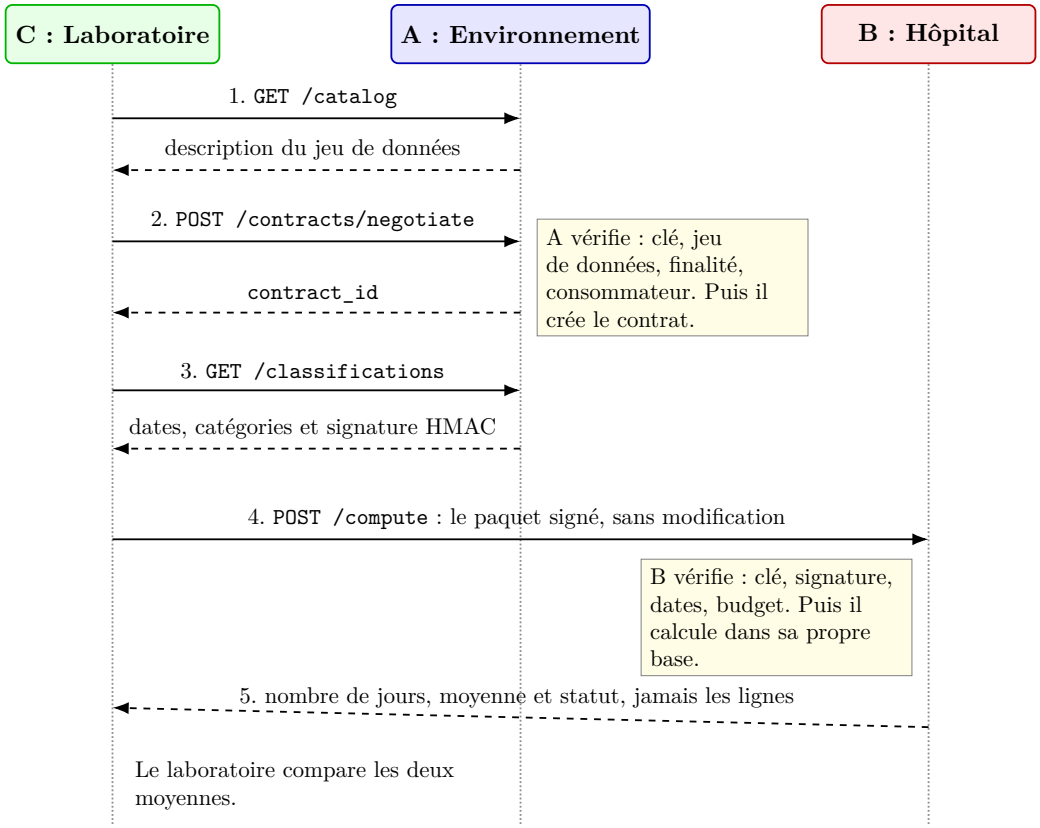



FIGURE 7.1 – Déroulement d'une analyse au TP 4

## 7.2 Mise en œuvre

**Données.** Le TP 4 a son propre dossier. `setup_data.py` crée deux bases avec les 24 communes où le PM2.5 est mesuré.

**Nœud A.** Avant de créer un contrat, A vérifie dans l'ordre : la clé (401), l'existence du jeu de données et de la commune (404), la finalité `research` (403) et le consommateur (403). Ensuite, il classe chaque jour (`au_dessus_seuil` ou `sous_seuil`) et signe la liste complète avec HMAC. La signature fonctionne comme un cachet officiel : si on change une seule ligne, elle n'est plus valable. La commune fait aussi partie de la signature, pour que le laboratoire ne puisse pas changer de ville.

```
message = json.dumps(
    {"contract_id": contract_id, "consumer_id": consumer_id,
     "ville": ville, "classifications": classifications},
    sort_keys=True, separators=(",", ":"), ensure_ascii=False,
).encode("utf-8")
return hmac.new(SECRET_FEDERATION, message, hashlib.sha256).hexdigest()
```

Extrait 7.1 – Signature de la liste par le nœud A

**Nœud B.** B vérifie la clé, la signature, la liste, les dates et le budget, puis calcule dans sa base. Un groupe de moins de 3 jours n'est pas publié (**seuil minimal**). Si on le demande, une petite erreur au hasard est ajoutée à la moyenne (**bruit de Laplace**), d'échelle $b = (\texttt{BORNE_MAX}/n)/\varepsilon$, où $n$ est le nombre de jours du groupe. `BORNE_MAX` vaut 350, car le maximum observé est de 341 admissions par jour. Le **budget** limite à 3 le nombre de calculs bruités par consommateur (avec $\varepsilon = 1$). Il est enregistré dans `hopital.db` : un nouveau contrat ou un redémarrage de l'API ne le remet donc pas à zéro. Sans cette limite, on pourrait répéter la question et faire la moyenne des réponses pour enlever le bruit.

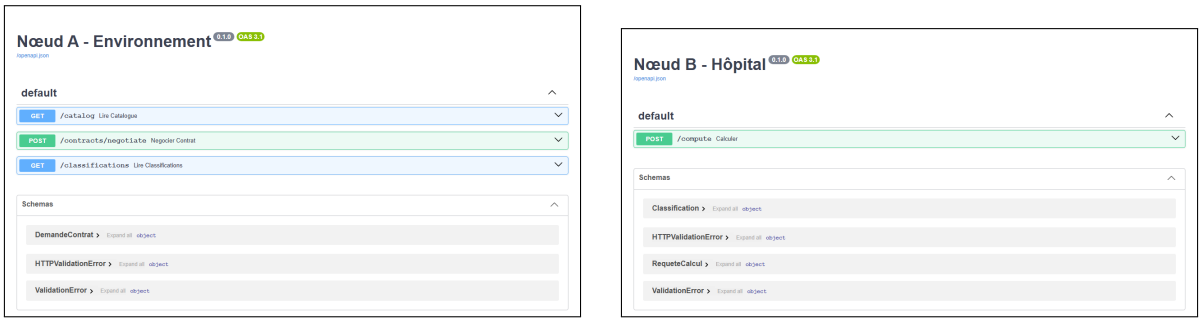



FIGURE 7.2 – Pages `/docs` des nœuds A et B (TP 4)

## 7.3 Résultats et tests

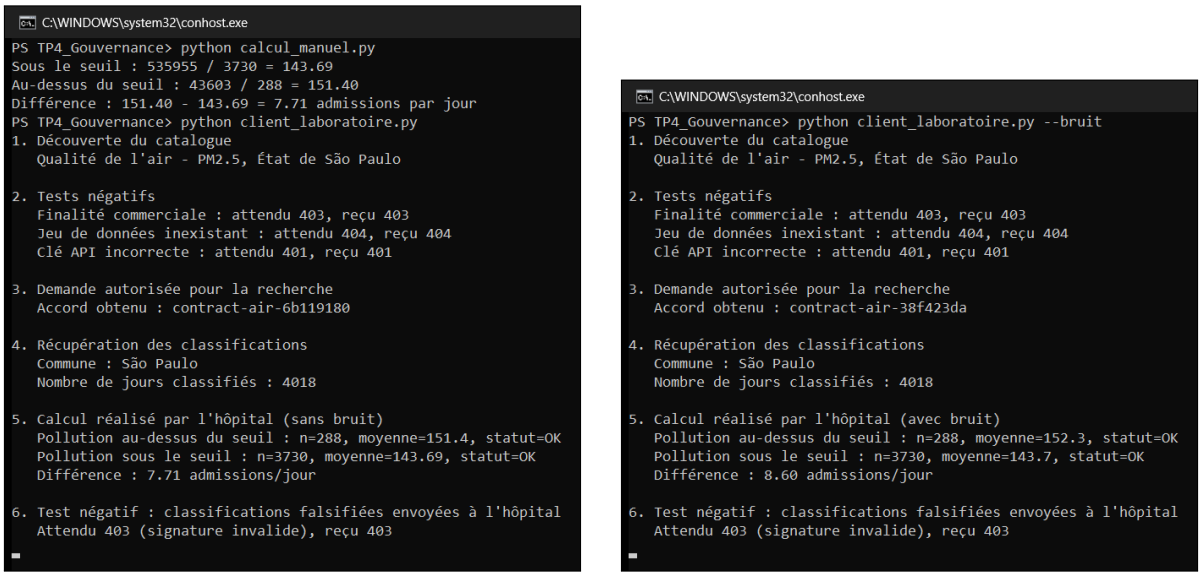



FIGURE 7.3 – Calcul manuel et scénario sans bruit (à gauche), scénario avec bruit (à droite)

**Calcul manuel**, fait avec `calcul_manuel.py` uniquement pour vérifier, car ce script ouvre les deux bases :

- Au-dessus du seuil : $43\,603/288 = 151{,}40$ admissions par jour ;
- Sous le seuil : $535\,955/3\,730 = 143{,}69$ admissions par jour ;
- Différence : $151{,}40 - 143{,}69 = 7{,}71$ admissions par jour.

Ces valeurs sont identiques à celles calculées par l'hôpital.

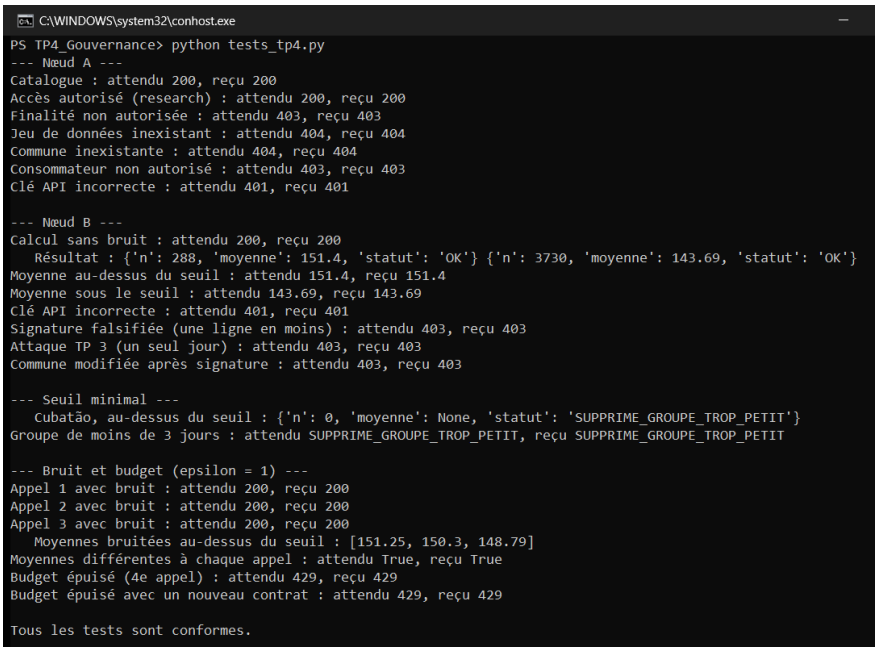



FIGURE 7.4 – Tests du TP 4 : finalité, clé, signature, attaque, seuil et budget

| **Test négatif** | **Attendu** | **Obtenu** |
|---|---|---|
| Finalité non autorisée (`commercial`) | 403 | 403 |
| Clé API incorrecte | 401 | 401 |
| Signature falsifiée (une ligne en moins) | 403 | 403 |
| Attaque du TP 3 (un seul jour) | 403 | 403 |
| Commune modifiée après signature | 403 | 403 |
| Jeu de données inexistant | 404 | 404 |
| Budget épuisé ($4^{e}$ appel avec bruit) | 429 | 429 |
| Cubatão : 0 jour au-dessus du seuil | groupe supprimé | groupe supprimé |

TABLE 7.1 – Tests négatifs du TP 4

## 7.4 Réponses aux questions

**Q1. Informations pour découvrir le jeu de données ?** Le catalogue : identifiant, titre, description, propriétaire, période, communes, unité, adresse d'accès et finalité autorisée.

**Q2. DCAT, et pourquoi une simplification ?** DCAT est un vocabulaire du W3C pour décrire des catalogues de données. Le catalogue du prototype est une simple liste JSON avec des champs voisins : pas de RDF ni de classes officielles, donc il n'est pas conforme.

**Q3. Pourquoi contrôler la finalité ? Catalogue, politique, contrôle d'accès ?** Une donnée peut être acceptable pour la recherche, mais pas pour un usage commercial. Le **catalogue** décrit ce qui existe, la **politique d'usage** dit ce qui est permis, et le **contrôle d'accès** applique la décision à chaque demande. Attention : la finalité est seulement déclarée par le laboratoire.

**Q4. Ce qui circule, ce qui reste ?** A envoie à C le catalogue, le contrat, les dates avec leur catégorie et la signature. C transmet ce paquet à B. B renvoie à C le nombre de jours,

la moyenne et le statut. Les admissions restent chez B et les concentrations exactes chez A. La conservation locale ne suffit pas, car ce qui sort peut permettre de retrouver une valeur (attaque du TP 3).

**Q5. Petits groupes, k-anonymat, signature ?** Dans un petit groupe, la moyenne est proche d'une valeur individuelle. Le k-anonymat demande qu'une personne soit impossible à distinguer d'au moins $k - 1$ autres ; ici, on compte des jours et non des personnes. La signature couvre toute la liste : le laboratoire ne peut plus choisir ses dates (403).

**Q6. Avec et sans bruit ?** Sans bruit : 151,40 et 143,69 (écart 7,71). Avec bruit ($\varepsilon = 1$), une exécution donne 152,3 et 143,7 (écart 8,60), et cela change à chaque fois. Un $\varepsilon$ petit protège plus, mais est moins précis. Ce n'est pas une implémentation complète : la borne vient des données, les effectifs ne sont pas bruités, les catégories de A ne sont pas protégées, et le calcul sans bruit reste possible.

**Q7. Peut-on conclure à une causalité ?** Non, c'est seulement une association. Il faudrait tenir compte de la saison, de la température, de l'humidité, des épidémies, du jour de la semaine, des autres polluants, de l'âge et de l'accès aux soins (voir le chapitre 8).

# 8. Résultats et discussion

**Mêmes résultats, moins d'informations révélées.** Les TP 2, 3 et 4 donnent les mêmes moyennes (151,40 et 143,69). Au TP 2, le laboratoire voit toutes les lignes ; au TP 3, il ne les voit plus ; au TP 4, il ne peut plus tricher.

**Une différence de 7,71 ne prouve pas une cause.** Les jours pollués ne tombent pas au hasard : sur 288 jours au-dessus de 30 $\mu$g/m$^3$, 271 sont entre mai et septembre, et le nombre d'admissions change aussi selon le mois.

| **Mois** | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| PM2.5 moyen | 12,0 | 12,2 | 12,9 | 14,2 | 16,6 | 19,3 | 22,3 | 20,2 | 21,2 | 14,2 | 12,1 | 12,3 |
| Admissions/jour | 99 | 121 | 162 | 178 | 167 | 160 | 140 | 145 | 141 | 145 | 145 | 127 |
| Jours > 30 | 0 | 1 | 0 | 6 | 17 | 45 | 76 | 67 | 66 | 9 | 1 | 0 |

TABLE 8.1 – Pollution et admissions selon le mois (commune de São Paulo, 2015–2025)

En comparant à l'intérieur de chaque mois de chaque année, l'écart est de 6,34. Sans les années 2020 et 2021, il est de 9,55. L'écart reste positif, mais il change selon le calcul. On voit aussi d'autres effets : 159,8 admissions le mardi contre 111,5 le dimanche, et une forte baisse en 2020 (107,9 par jour). **Il y a donc une association, mais ces calculs ne prouvent pas que la pollution en est la cause.**

**Effet du bruit.** Avec $\varepsilon = 1$, l'écart-type du bruit vaut 1,72 pour le groupe de 288 jours et 0,13 pour celui de 3 730 jours. Avec un groupe de 5 jours et une borne de 30, il serait d'environ 8,5, soit plus que la différence observée ici. Une grande base permet de protéger le résultat en gardant une réponse utile.

**Limites du prototype.** Les clés et le secret sont des valeurs de démonstration écrites dans le code. Les échanges sont en HTTP, sans chiffrement. Le secret HMAC est partagé, donc B pourrait aussi signer. Le `consumer_id` est seulement déclaré. Les contrats sont gardés en mémoire et disparaissent au redémarrage de l'API.

## 8.1 Limites

— **Calcul sans bruit :** le laboratoire peut encore demander un résultat sans bruit (`apply_dp=False`), ce qui ne consomme pas le budget $\varepsilon$. Cette option est conservée pour comparer les résultats avec et sans bruit pendant la démonstration. En conditions réelles, l'hôpital devrait imposer le bruit côté serveur.

— **Données PM2.5 incomplètes :** les mesures ne couvrent que 24 communes, alors que la base hospitalière des TP 1 à 3 en contient 645 (celle du TP 4 ne garde que les 24 communes mesurées), et certains jours peuvent manquer. Les valeurs absentes n'ont pas été complétées artificiellement : l'analyse porte uniquement sur les couples commune-jour réellement mesurés, et ses résultats ne représentent donc pas tout l'État de São Paulo. Le jeu source contient déjà des moyennes journalières, calculées avec les filtres de complétude décrits dans son README (au moins 18 valeurs horaires valides par jour).

# Conclusion générale

Ce mini-projet montre comment construire, étape par étape, un petit espace de données entre trois organisations, sur une base réelle de plusieurs millions de lignes. Il met en évidence la différence entre le **stockage fédéré** (garder les données dans des bases séparées, TP 1), le **traitement fédéré** (calculer là où sont les données, TP 3 et 4) et l'**analyse fédérée** (combiner des résultats autorisés sans accéder aux bases, TP 3 et 4).

Il montre aussi qu'une API et une clé ne suffisent pas. Le TP 3 protégeait l'accès, mais une simple attaque a retrouvé la valeur exacte d'un jour. Le TP 4 corrige ces limites :

| **Limite du TP 3** | **Correction du TP 4** | **Preuve** |
|---|---|---|
| Le labo choisit ses groupes | Liste complète signée par A | Attaque refusée (403) |
| Une clé API suffit | Contrat et finalité | `commercial` refusé (403) |
| Pas de découverte des données | Catalogue | `GET /catalog` (200) |
| Petits groupes publiés | Seuil de 3 jours | Cubatão supprimé |
| Résultats exacts répétables | Bruit et budget | 4$^{e}$ appel refusé (429) |

TABLE 8.2 – Ce que le TP 4 corrige

Ce prototype reste un exercice : un vrai système demanderait du HTTPS, de vraies identités, une gestion sécurisée des secrets, des journaux d'accès et les vraies normes DCAT et ODRL. Une suite logique serait d'ajouter un modèle statistique et d'analyser plusieurs communes.

# A. Glossaire des abréviations

| Terme | Signification | Terme | Signification |
|---|---|---|---|
| PM2.5 | Particules fines de moins de 2,5 micromètres | API | Interface pour demander des données à un programme |
| $\mu g/m^3$ | Microgrammes par mètre cube | REST | Façon standard d'organiser une API |
| J00–J99 | Codes médicaux respiratoires | SQL | Langage pour interroger une base |
| CETESB | Agence de l'environnement de São Paulo | HTTP(S) | Protocole du web (S = chiffré) |
| SUS | Système de santé public du Brésil | JSON | Format de texte pour échanger des données |
| SIH | Système d'information hospitalier du SUS | Port | Numéro d'un service sur une machine |
| DATASUS | Service qui publie les données du SUS | Parquet | Format pour grands tableaux de données |
| DCAT | Vocabulaire du W3C pour les catalogues | MD5 | Empreinte pour vérifier un fichier |
| ODRL | Langage du W3C pour les règles d'usage | W3C | Organisme des normes du web |
| HMAC | Signature avec un secret partagé | $\varepsilon$ | Réglage du bruit (petit = plus protégé) |

TABLE A.1 – Abréviations utilisées

**Codes de réponse :** 200 réussi, 400 demande mal formulée, 401 clé absente ou incorrecte, 403 accès interdit, 404 introuvable, 429 budget épuisé.

# Références

[1] Support de travaux pratiques, *Mini-espace de données fédéré – Santé et qualité de l'air*, Master IASD.

[2] J. R. L. F. Arcoverde, C. D. Maciel, *Air quality and respiratory hospitalizations in São Paulo State, Brazil*, Zenodo, 2026. `https://doi.org/10.5281/zenodo.23164494`

[3] W3C, *Data Catalog Vocabulary (DCAT)*. `https://www.w3.org/TR/vocab-dcat-3/`

[4] W3C, *ODRL Information Model 2.2*. `https://www.w3.org/TR/odrl-model/`

[5] Documentation FastAPI. `https://fastapi.tiangolo.com/`

[6] H. Krawczyk, M. Bellare, R. Canetti, *HMAC : Keyed-Hashing for Message Authentication*, RFC 2104, 1997.

[7] C. Dwork, A. Roth, *The Algorithmic Foundations of Differential Privacy*, 2014.

[8] L. Sweeney, *k-anonymity : a model for protecting privacy*, 2002.